ELO320 - Estructuras de Datos y Algoritmos
Listas Doblemente Enlazadas

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

1er Semestre 2025

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

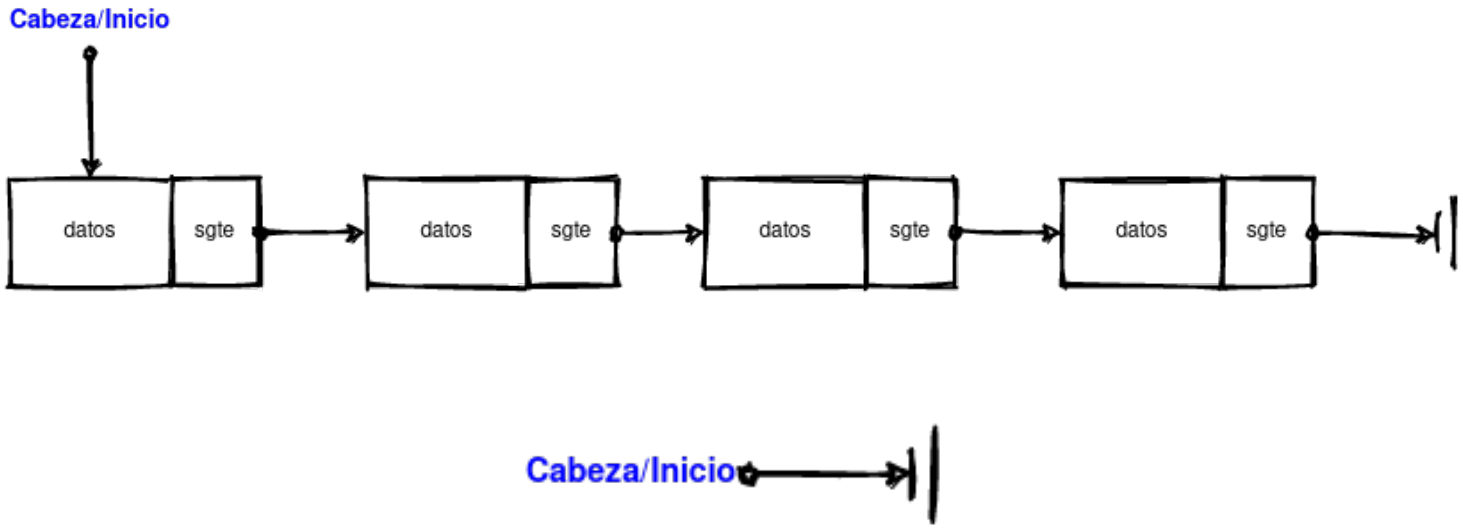
Lista Simple Enlazada: Inicializar

Las listas, en su versión más básica, dependen de un puntero al inicio de la lista. Éste suele llamarse *header*, *cabecera* o *inicio*.

C

```
#include<stdio.h>

typedef struct nodo{
        int dato; //dato, puede ser lo que uds. necesiten
        struct nodo *sgte; // puntero a un nodo enalce.
} nodo_t;

int main(int argc, char **argv){

        nodo_t * inicio = NULL;
        // Inicializamos la lista, con su puntero de inicio apuntando a NULL
        // Equivalente a lista vacía []

        return 0;
}
```

Lista Simple Enlazada: Inicializar



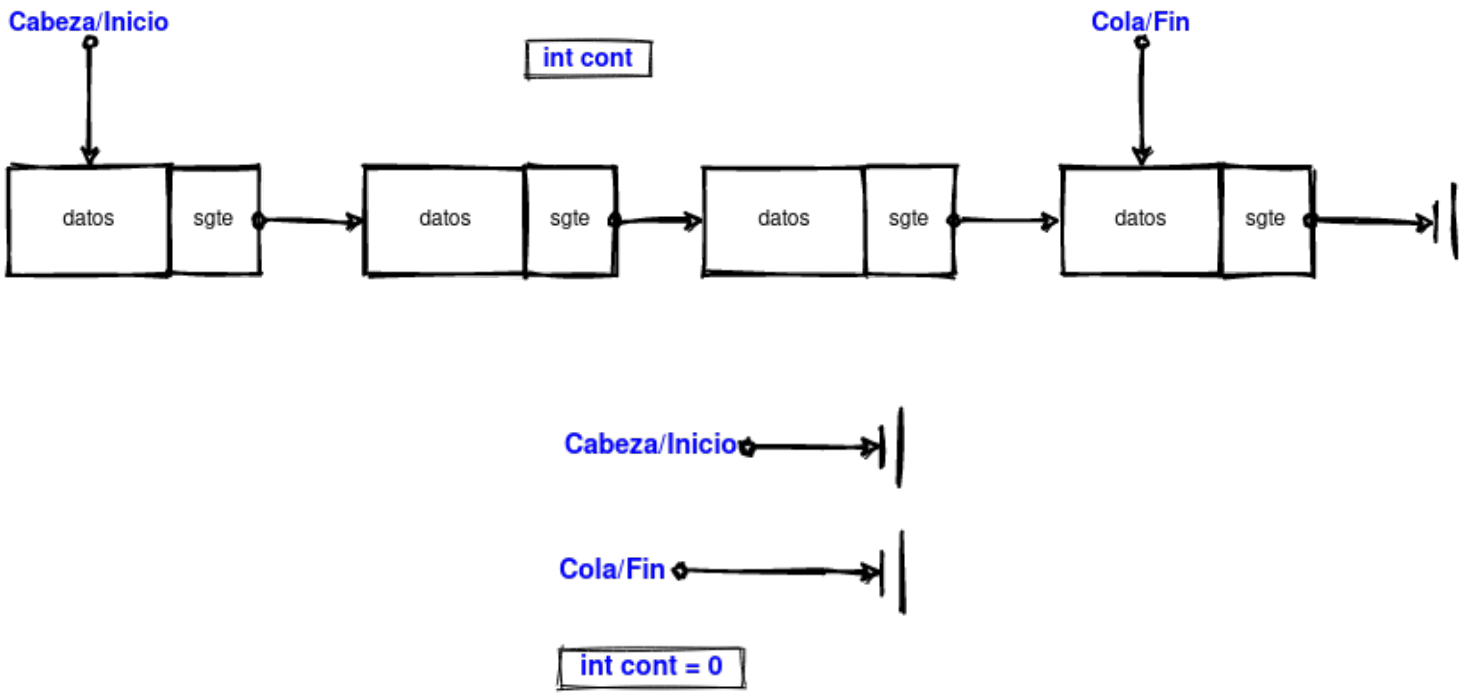
Lista Enlazada PRO

Existen otras implementaciones de listas enlazadas simples, donde se incluyen más punteros y variables de control.

C

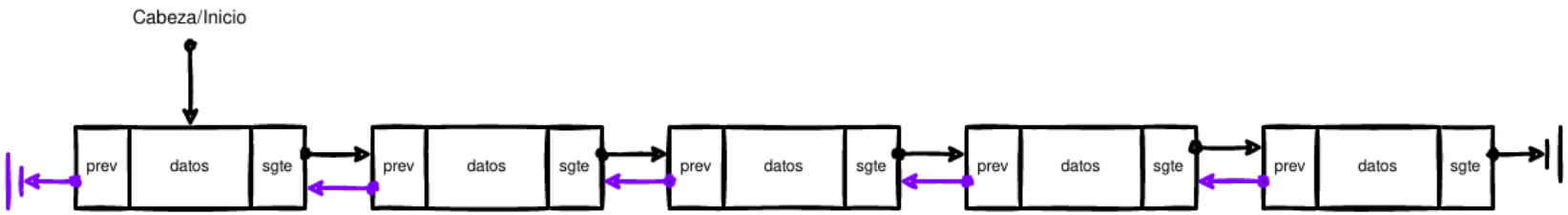
```
#include<stdio.h>

typedef struct nodo{
        int dato; //dato, puede ser lo que uds. necesiten
        struct nodo *sgte; // puntero a un nodo enalce.
} nodo_t;

int main(int argc, char **argv){

        nodo_t * inicio = NULL; //pueden usar un struct para estos datos
        nodo_t * final = NULL;
        int largo = 0;
        // Equivalente a lista vacía []
        // ¿Para que sirve esta nueva forma?
        // Implemente las funciones de ésta clase considerando esta forma.

        return 0;
}
```

Lista Enlazada PRO



Lista Doblemente Enlazada

Una lista doblemente enlazada es una modificación de la lista simple enlazada. Éste TDA tiene un enlace más, para acceder rápidamente al nodo previo y tener una mejor nagevación sobre la estructura.



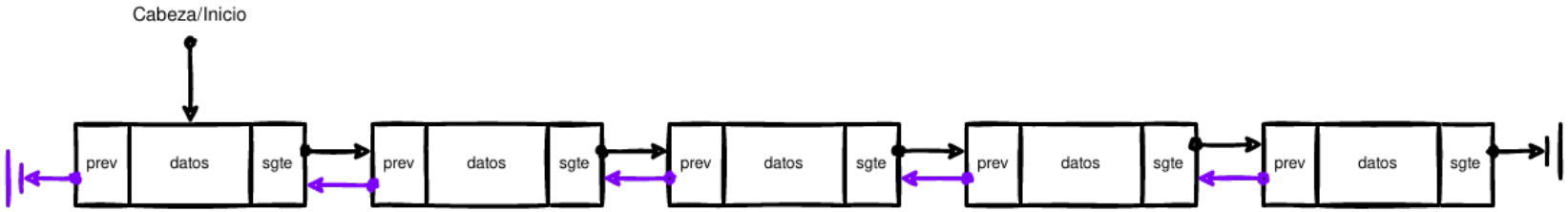
Está compuesta por:

- Nodos: Contienen la información en lugares de memoria no ordenados.
- Punteros: Enlazan los distintos nodos para conformar la lista.

Lista Doblemente Enlazada

Los punteros permiten al menos:

- Controlar el inicio de la lista.
- Apuntar al siguiente elemento en la lista.
- Apuntar al anterior elemento en la lista.
- Acceder a cada nodo.



Lista Doblemente Enlazada: Nodos

Los nodos son definidos con un struct, que almenos contiene un tipo de dato y un puntero.

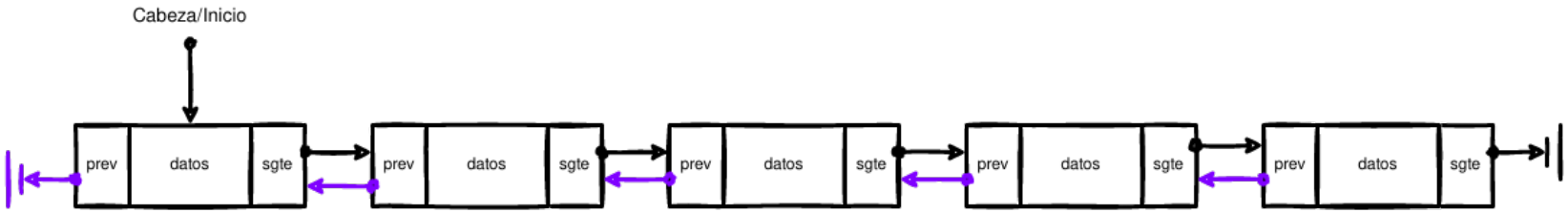
C

```
struct nodo{
        int dato; //dato, puede ser lo que uds. necesiten
        struct nodo *sgte; // puntero a un nodo siguiente, enlace
        struct nodo *prev; // puntero a un nodo previo, enlace

};

typedef struct le_nodo{
        float datos[20]; //dato, puede ser lo que uds. necesiten
        struct le_nodo *sgte; // puntero a un nodo siguiente, enalce.
        struct le_nodo *prev; // puntero a un nodo previo, enalce.

} nodo_t;
```

Lista Doblemente Enlazada: Interfaz

Al igual que las listas simples enlazadas tienen un conjunto de funciones que interactúan como interfaz:

- Inicializar Lista.
- Agregar un elemento a una lista.
- Determinar el largo de una lista.
- Eliminar un elemento de una lista.
- Imprimir una lista.
- Limpiar una lista.



Lista Doblemente Enlazada: Inicializar

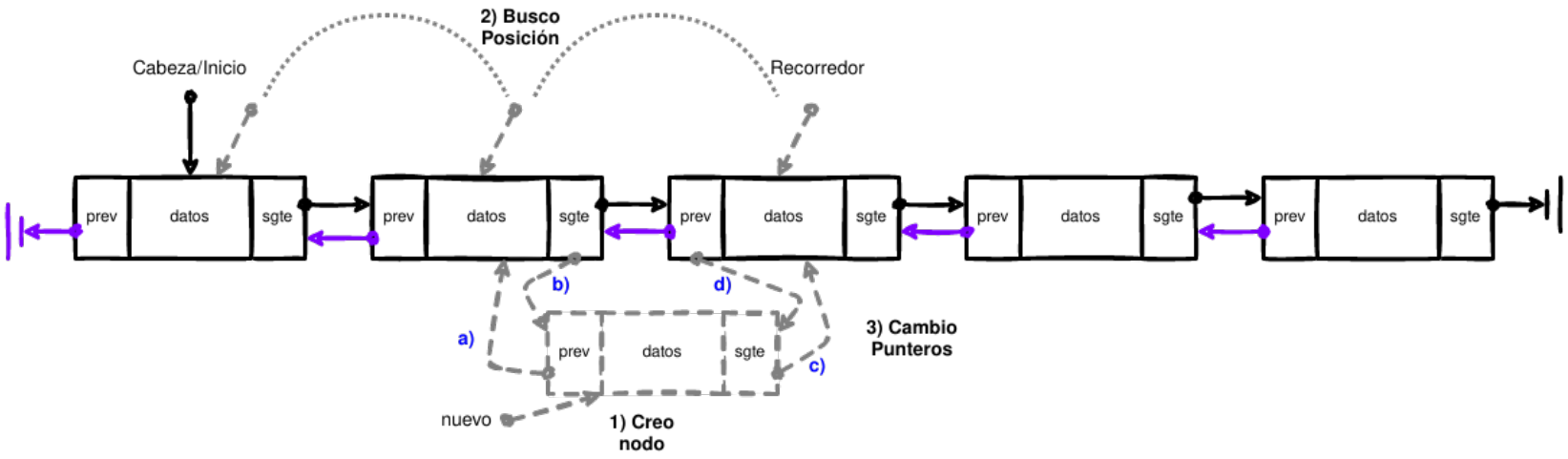
Las listas, en su versión más básica, dependen de un puntero al inicio de la lista. Éste suele llamarse *header*, *cabecera* o *inicio*. Las Listas Doblemente Enlazadas **siguen la misma dinámica** que las Listas Simple Enlazada.

C

```c
#include<stdio.h>

typedef struct nodo{
        int dato; //dato, puede ser lo que uds. necesiten
        struct nodo *sgte; // puntero a un nodo siguiente, enalce.
        struct nodo *prev; // puntero a un nodo previo, enalce.
} nodo_t;

int main(int argc, char **argv){

        nodo_t * inicio = NULL;
        // Inicializamos la lista, con su puntero de inicio apuntando a NULL
        // Equivalente a lista vacía []

        return 0;
}
```

Lista Doblemente Enlazada: Añadir nodos

Esquema General:

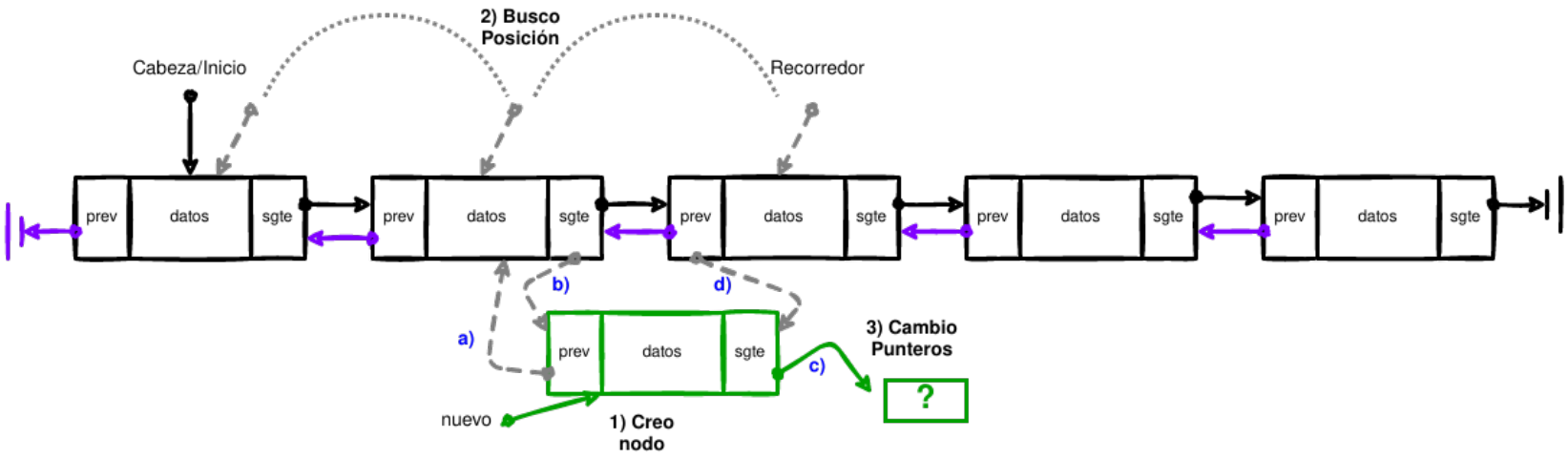


Debemos:

1. Crear el nodo `nuevo` y asignar el dato.
2. Buscar la posición a agregar el nodo.
3. Cambiar los punteros.

Lista Doblemente Enlazada: Añadir nodos

Esquema General:

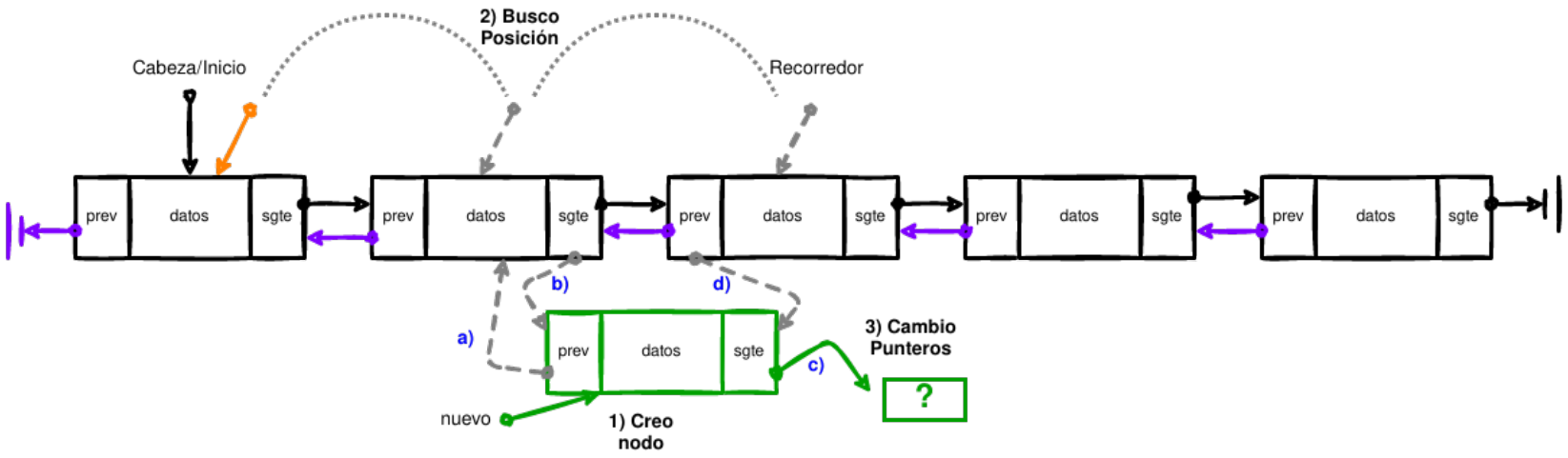


Debemos:

1. Crear el nodo `nuevo` y asignar el dato.
2. Buscar la posición a agregar el nodo.
3. Cambiar los punteros.

Lista Doblemente Enlazada: Añadir nodos

Esquema General:

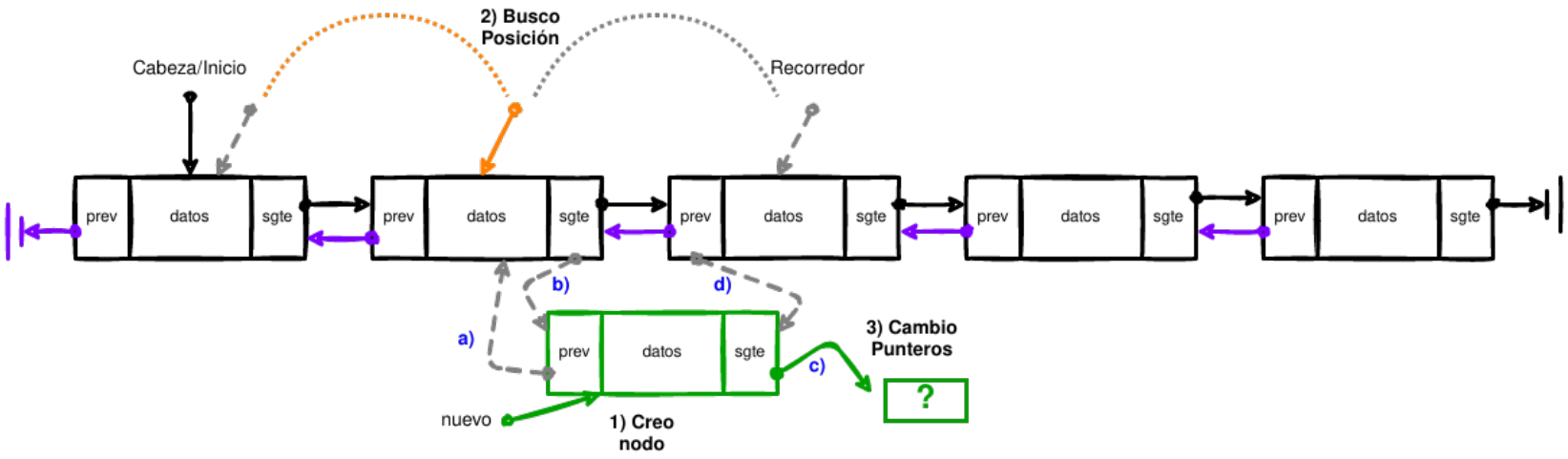


Debemos:

1. Crear el nodo `nuevo` y asignar el dato.
2. Buscar la posición a agregar el nodo.
3. Cambiar los punteros.

Lista Doblemente Enlazada: Añadir nodos

Esquema General:

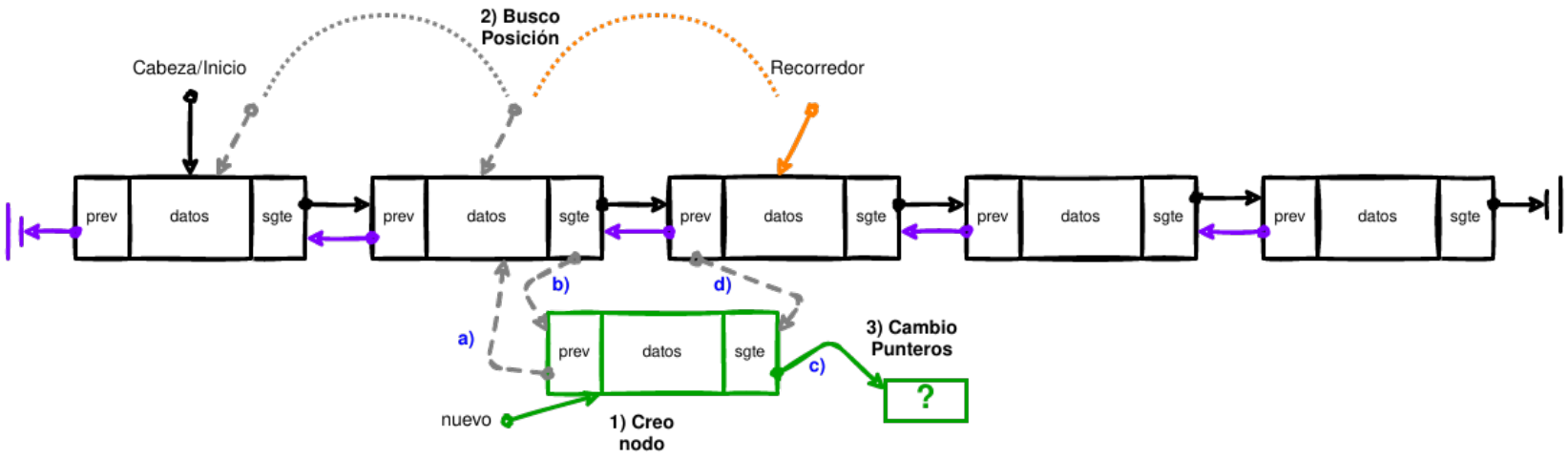


Debemos:

1. Crear el nodo `nuevo` y asignar el dato.
2. Buscar la posición a agregar el nodo.
3. Cambiar los punteros.

Lista Doblemente Enlazada: Añadir nodos

Esquema General:

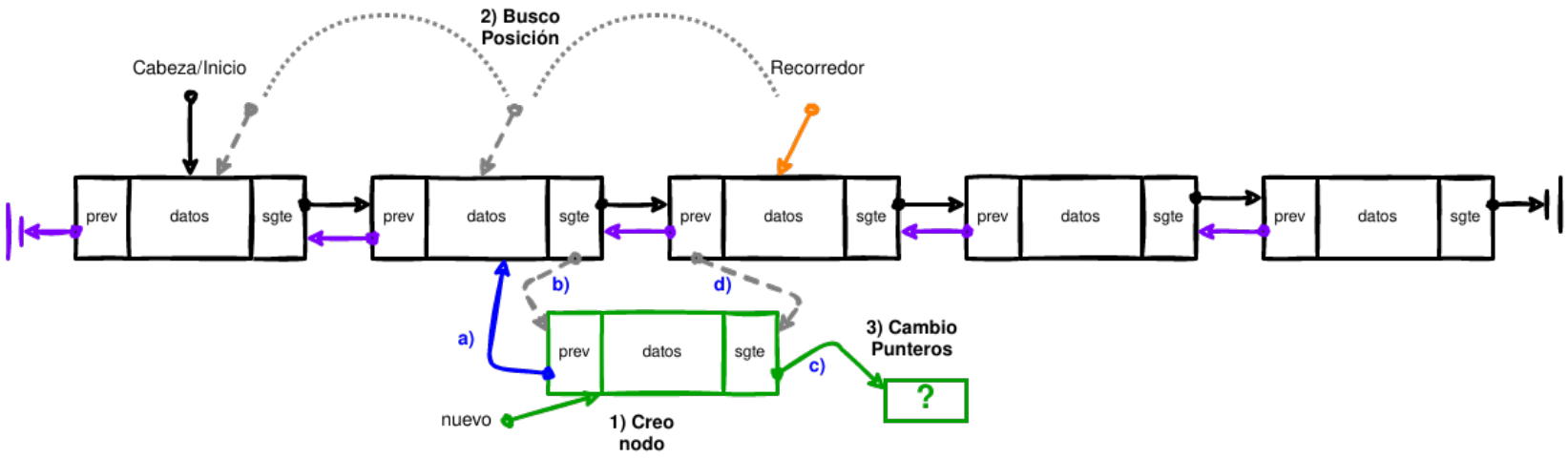


Debemos:

1. Crear el nodo `nuevo` y asignar el dato.
2. Buscar la posición a agregar el nodo.
3. Cambiar los punteros.

Lista Doblemente Enlazada: Añadir nodos

Esquema General:

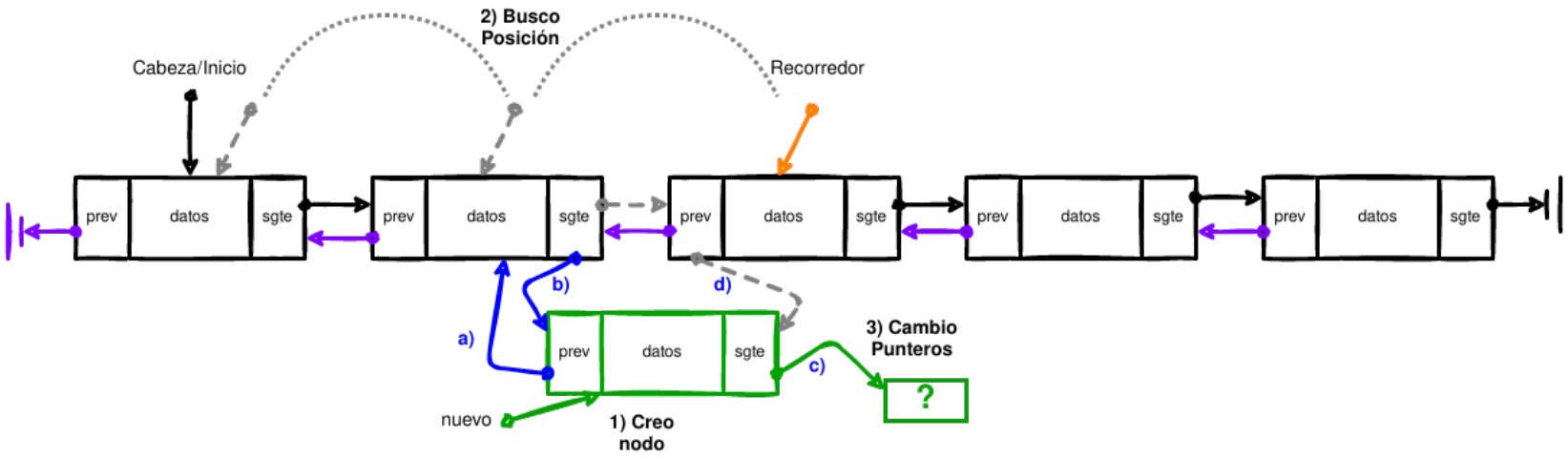


Debemos:

1. Crear el nodo `nuevo` y asignar el dato.
2. Buscar la posición a agregar el nodo.
3. Cambiar los punteros.

Lista Doblemente Enlazada: Añadir nodos

Esquema General:

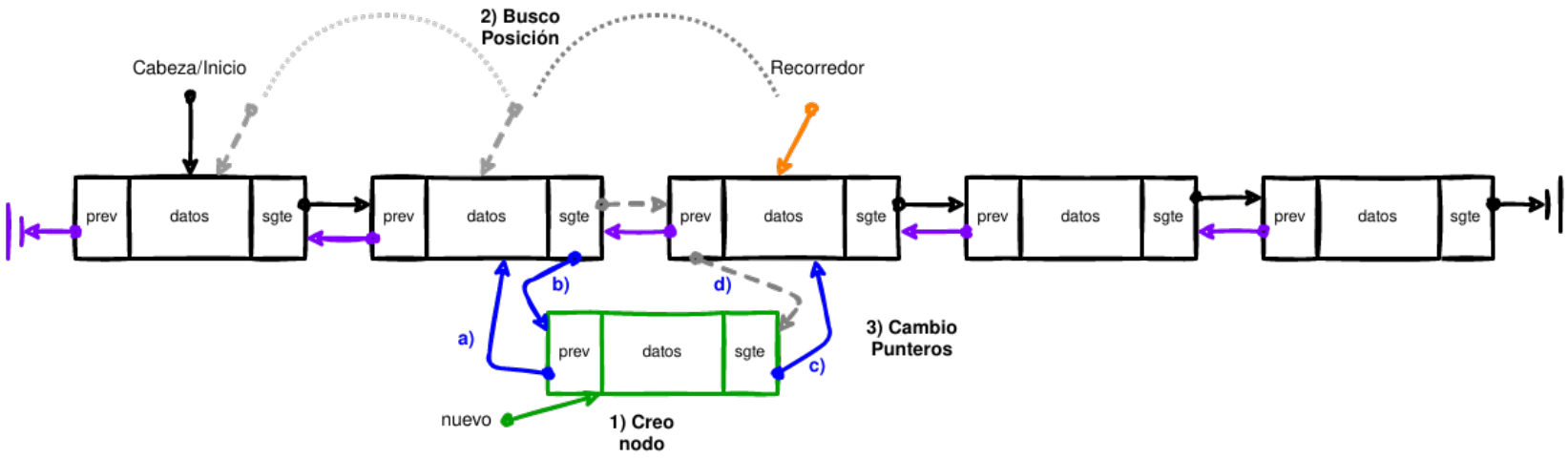


Debemos:

1. Crear el nodo `nuevo` y asignar el dato.
2. Buscar la posición a agregar el nodo.
3. Cambiar los punteros.

Lista Doblemente Enlazada: Añadir nodos

Esquema General:

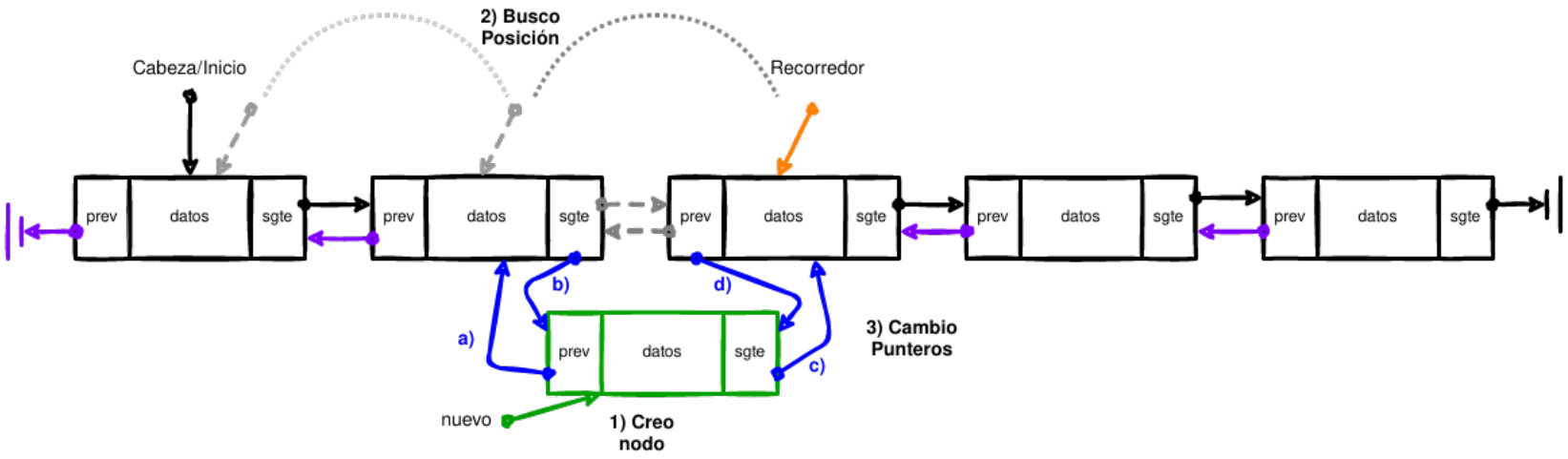


Debemos:

1. Crear el nodo `nuevo` y asignar el dato.
2. Buscar la posición a agregar el nodo.
3. Cambiar los punteros.

Lista Doblemente Enlazada: Añadir nodos

Esquema General:

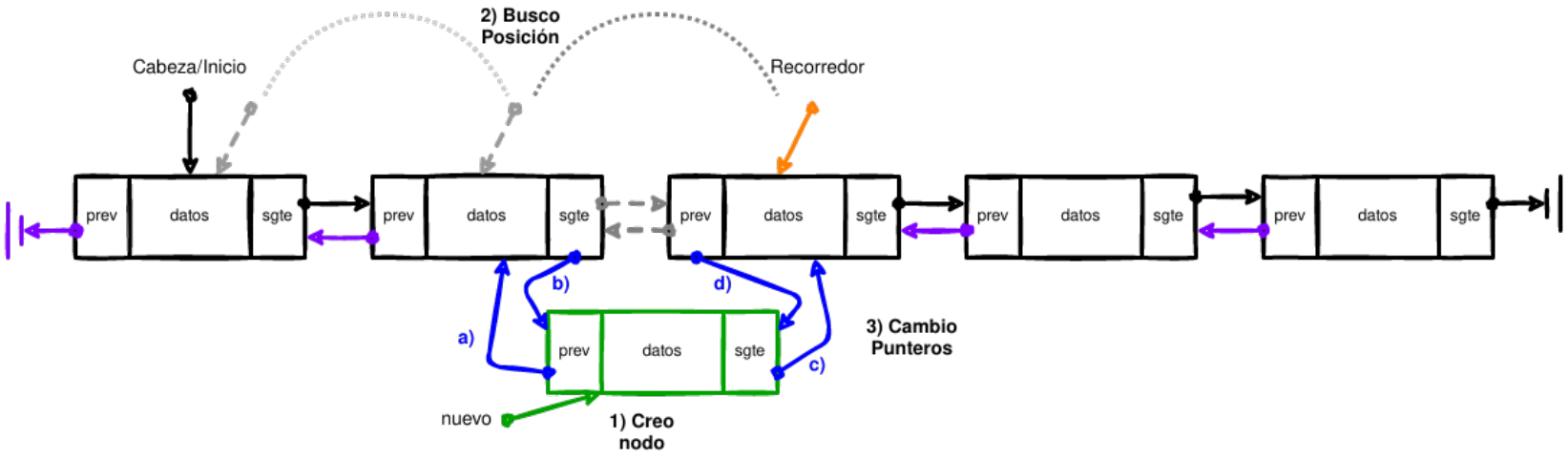


Debemos:

1. Crear el nodo `nuevo` y asignar el dato.
2. Buscar la posición a agregar el nodo.
3. Cambiar los punteros.

Lista Doblemente Enlazada: Añadir nodos

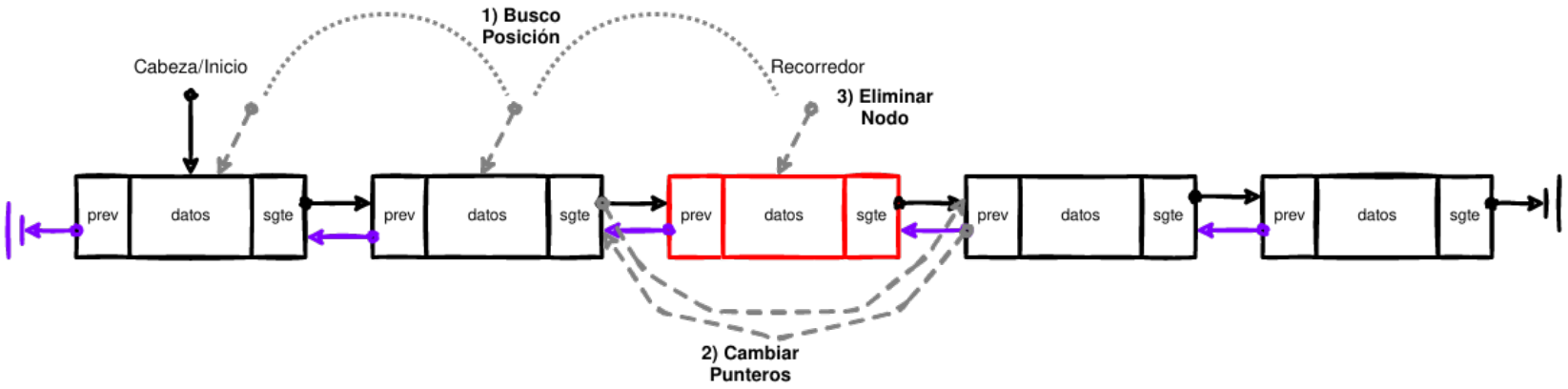
Esquema General:



- Implemente esta función.
- ¿Qué precaución hay que tener si se quiere agregar al inicio o al final de la lista?

Lista Doblemente Enlazada: Eliminar nodos

Esquema General:

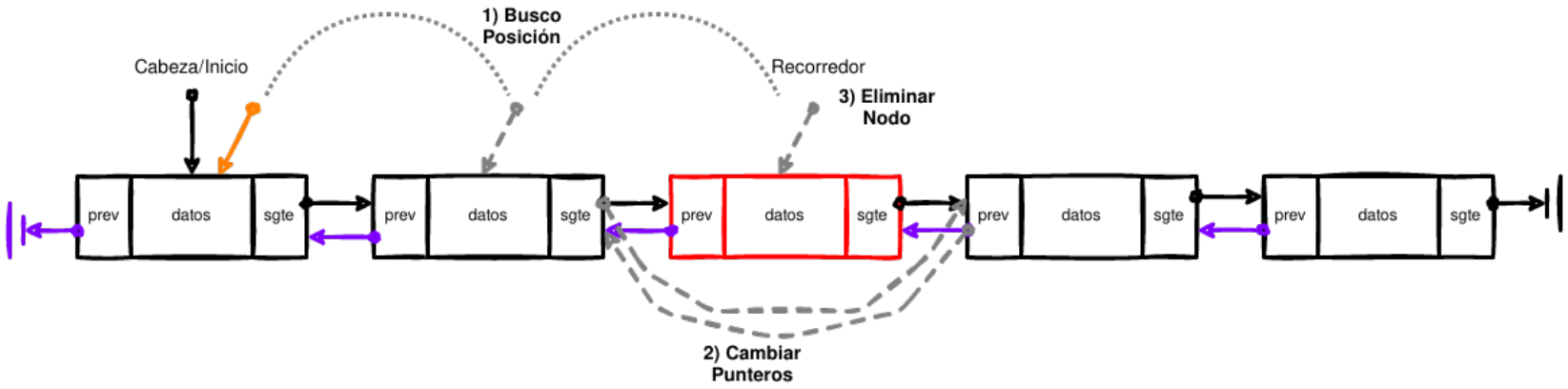


Ojo, que no siempre se pueden borrar nodos: **lista vacía**.
Debemos:

1. Buscar la posición del nodo a eliminar.
2. Cambiar los punteros.
3. Eliminar el nodo.

Lista Doblemente Enlazada: Eliminar nodos

Esquema General:

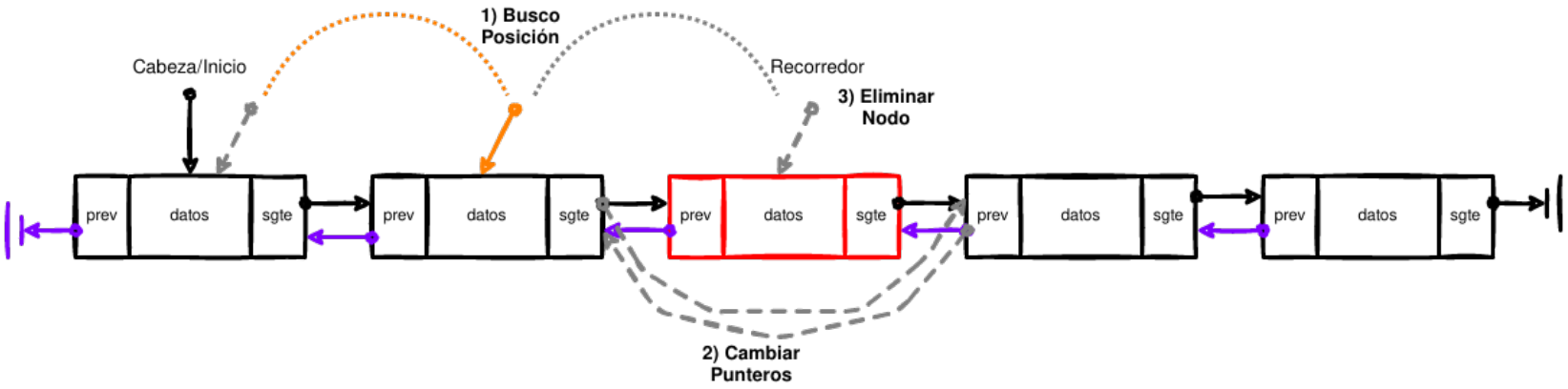


Ojo, que no siempre se pueden borrar nodos: **lista vacía**.
Debemos:

1. Buscar la posición del nodo a eliminar.
2. Cambiar los punteros.
3. Eliminar el nodo.

Lista Doblemente Enlazada: Eliminar nodos

Esquema General:

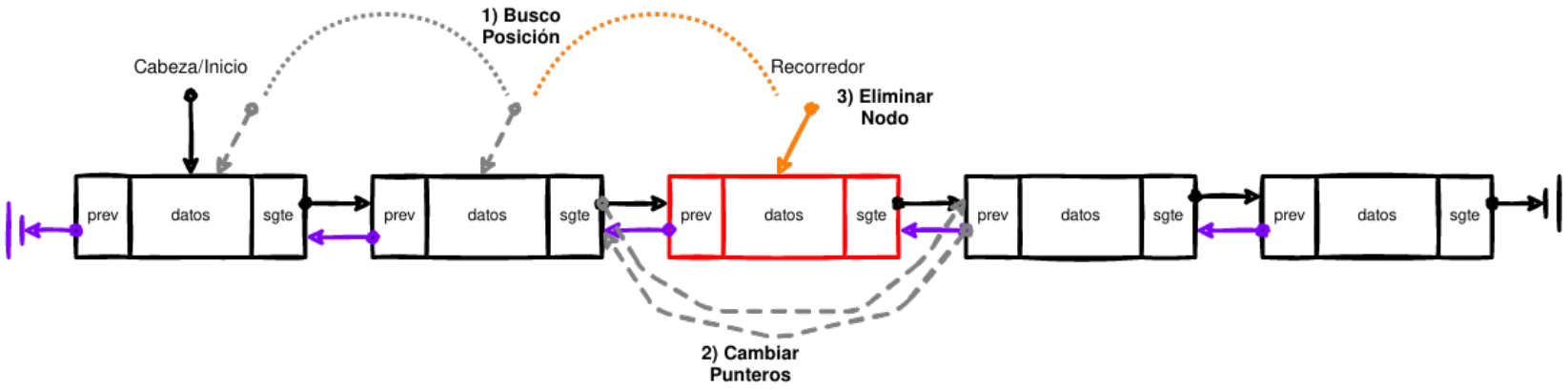


Ojo, que no siempre se pueden borrar nodos: **lista vacía**.
Debemos:

1. Buscar la posición del nodo a eliminar.
2. Cambiar los punteros.
3. Eliminar el nodo.

Lista Doblemente Enlazada: Eliminar nodos

Esquema General:

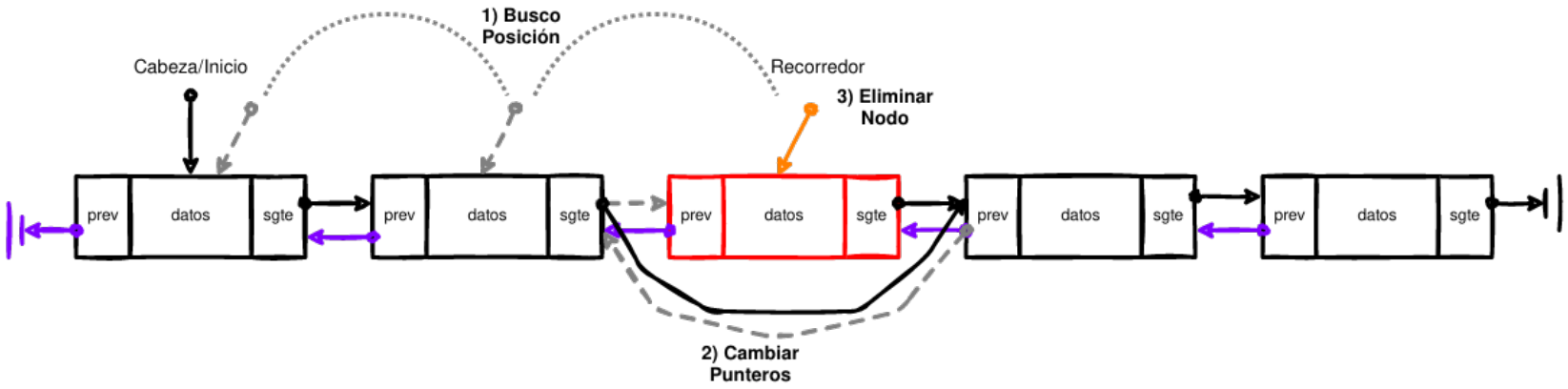


Ojo, que no siempre se pueden borrar nodos: **lista vacía**.
Debemos:

1. Buscar la posición del nodo a eliminar.
2. Cambiar los punteros.
3. Eliminar el nodo.

Lista Doblemente Enlazada: Eliminar nodos

Esquema General:

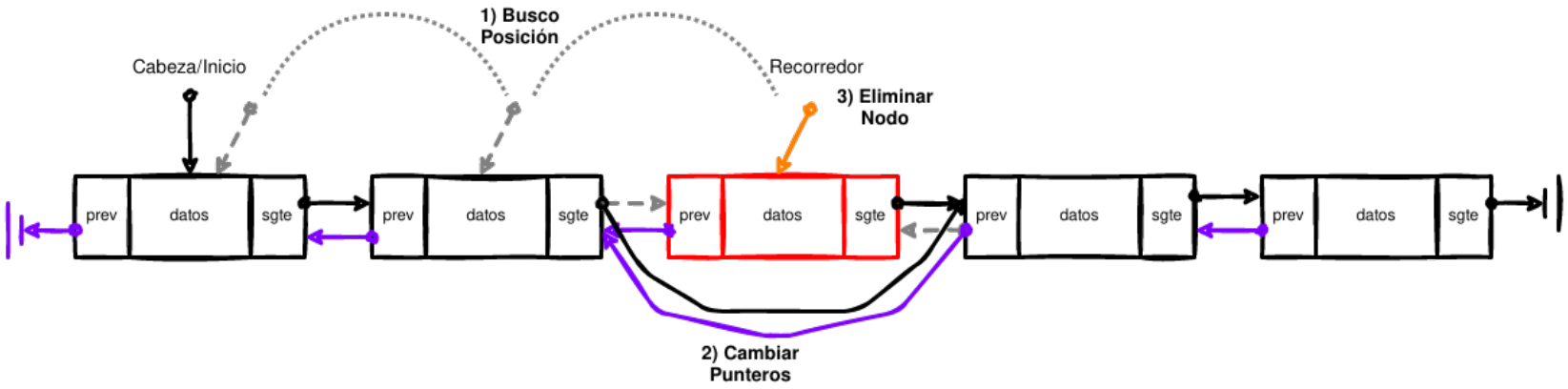


Ojo, que no siempre se pueden borrar nodos: **lista vacía**.
Debemos:

1. Buscar la posición del nodo a eliminar.
2. Cambiar los punteros.
3. Eliminar el nodo.

Lista Doblemente Enlazada: Eliminar nodos

Esquema General:

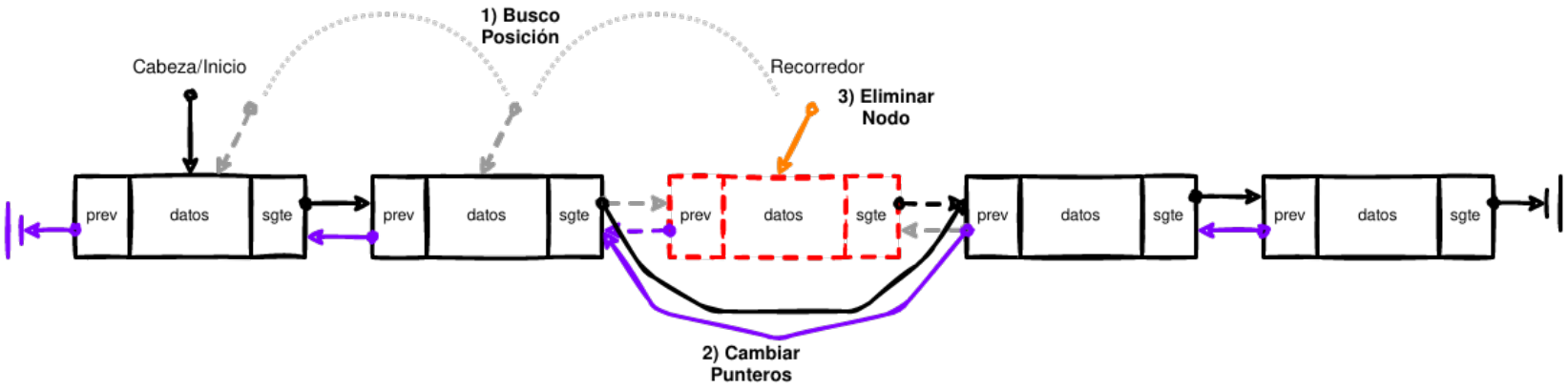


Ojo, que no siempre se pueden borrar nodos: **lista vacía**.
Debemos:

1. Buscar la posición del nodo a eliminar.
2. Cambiar los punteros.
3. Eliminar el nodo.

Lista Doblemente Enlazada: Eliminar nodos

Esquema General:

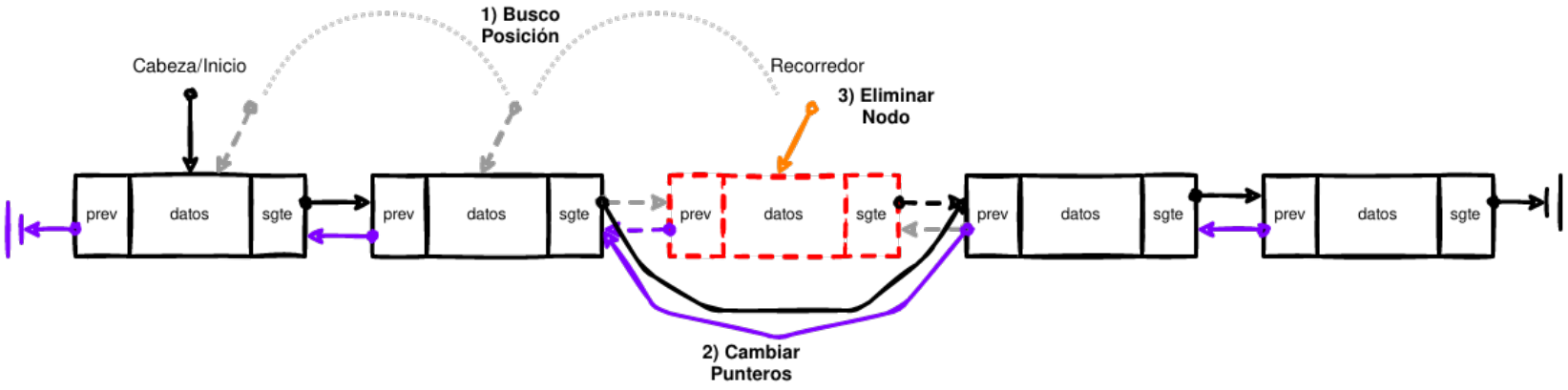


Ojo, que no siempre se pueden borrar nodos: **lista vacía**.
Debemos:

1. Buscar la posición del nodo a eliminar.
2. Cambiar los punteros.
3. Eliminar el nodo.

Lista Doblemente Enlazada: Eliminar nodos

Esquema General:



- Implemente esta función.
- ¿Qué precaución hay que tener si se quiere agregar al inicio o al final de la lista?

Lista Doblemente Enlazada: Funciones.

Basado en las conocidas funciones de Listas Simples Enlazadas, implemente las siguientes funciones para Listas Doblemente enlazadas:

- Determinar el largo de una lista.
- Imprimir una lista.
- Limpiar una lista.

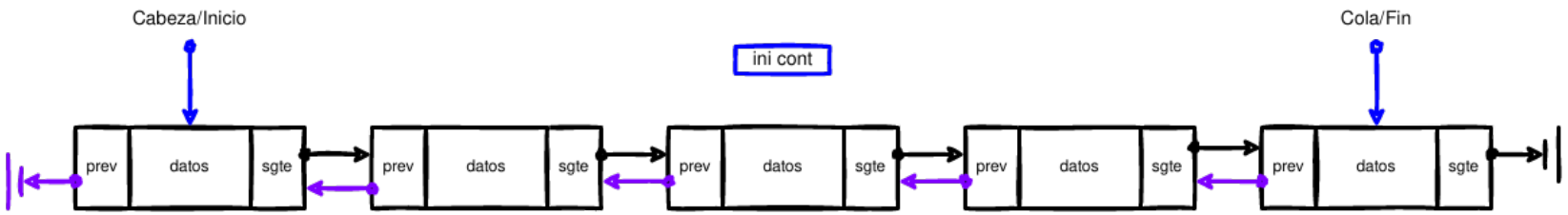
Lista Doblemente Enlazada PRO

Existen otras implementaciones de listas enlazadas simples, donde se incluyen más punteros y variables de control.

C

```
#include<stdio.h>

typedef struct nodo{
        int dato; //dato, puede ser lo que uds. necesiten
        struct nodo *sgte; // puntero a un nodo enalce.
} nodo_t;

int main(int argc, char **argv){

        nodo_t * inicio = NULL; //pueden usar un struct para estos datos
        nodo_t * final = NULL;
        int largo = 0;
        // Equivalente a lista vacía []
        // ¿Para que sirve esta nueva forma?
        // Implemente las funciones de ésta clase considerando esta forma.

        return 0;
}
```

Lista Doblemente Enlazada PRO



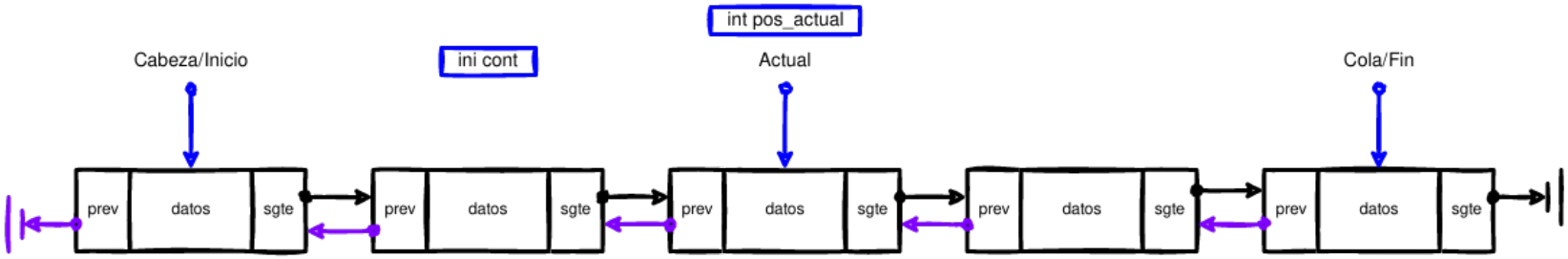
Lista Doblemente Enlazada PRO+

Existen otras implementaciones de listas enlazadas simples, donde se incluyen más punteros y variables de control.

C

```
#include<stdio.h>

typedef struct nodo{
        int dato; //dato, puede ser lo que uds. necesiten
        struct nodo *sgte; // puntero a un nodo enalce.
} nodo_t;

int main(int argc, char **argv){

        nodo_t * inicio = NULL; //pueden usar un struct para estos datos
        nodo_t * final = NULL;
        nodo_t * actual = NULL;
        int largo = 0;
        int pos_actual = 0;
        // Equivalente a lista vacía []
        // ¿Para que sirve esta nueva forma?
        // Implemente las funciones de ésta clase considerando esta forma.

        return 0;
}
```

Lista Doblemente Enlazada PRO+



Lista de Python en C

Implemente una lista de cada tipo; Simple, Simple PRO, Doble, Doble PRO, Doble PRO+; para algún tipo de dato a su elección, que incluya las funciones más conocidas de las listas de Python:

- `list()` o [].
- `.insert(pos, value)`
- `.append(value)`
- `.clear()`
- `del lista[pos]`
- `.remove(value)`
- `.sort()`
- `.reverse()`
- `.count(value)`
- `.index(value)`